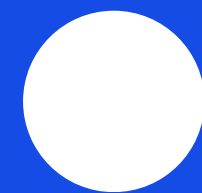


Codeit Sprint [AI] 초급 프로젝트

경구약제 이미지 인식

PillaTech



안수진, 김한별, 원숙현, 황예원

목차

01

프로젝트 개요

02

EDA

03

베이스라인 모델 선정

04

학습 단계 최적화

05

추론 성능 극대화

06

결론 및 향후 과제

프로젝트 개요

프로젝트 개요

복용 약물 증가에 따른
오남용 위험 및 정보 접근성 불균형 해소

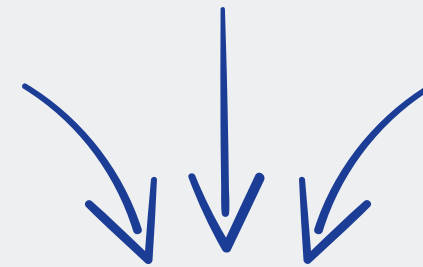
Problem

헬스케어 스타트업 'Health Eat'의
실시간 알약 식별 솔루션 개발

Mission

복용 데이터 기반 맞춤형 케어 및
비용 금기 약물 가이드 제공

Business Value



High-Precision & mAP Optimization

mAP@0.75:0.95 지표를 기반으로
오탐율을 최소화한 고정밀 알약 검출 구현

On-Device Latency Control

모바일 환경의 리소스 제한 고려하여
실시간 추론 속도와 모델 경량화 달성

Multi-Object Robustness

단일 이미지 내 최대 4종의 객체에 대한
독립적 식별 및 바운딩 박스 최적화

데이터셋 정의 및 구조

1. 데이터셋 개요 (Overview)

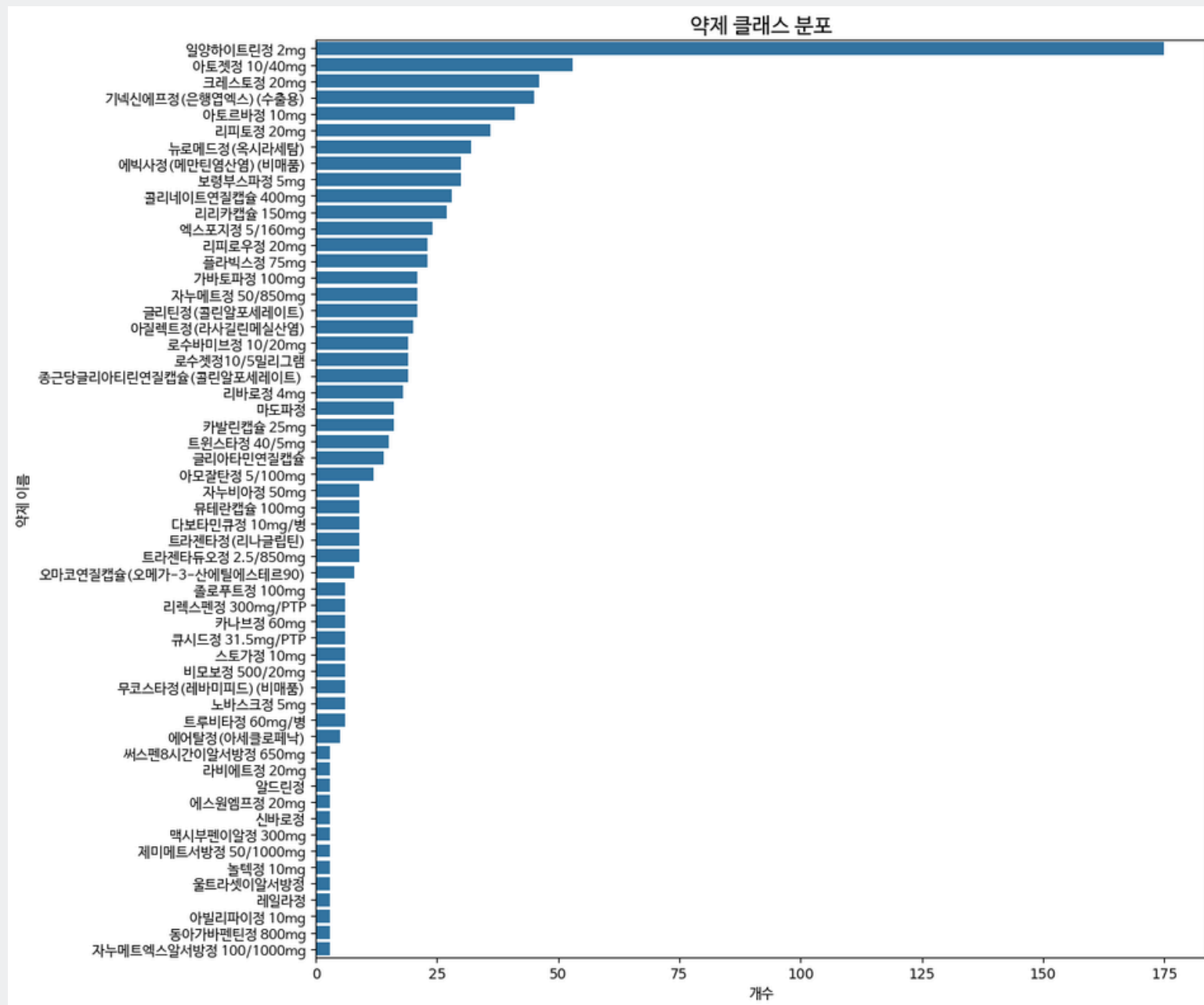
- Source: **AI hub 경구약제 이미지** 데이터셋 (코드잇에서 가공 후 제공)
- Scale: 총 -000장의 이미지 / 000종의 알약 클래스

2. 원본 데이터 구조 (Raw Data Structure)



EDA

탐색적 데이터 분석 (EDA Insight) – 클래스 분포



- 특정 클래스 편중 및 최소 데이터(3개) 클래스 다수 존재
- 전략

1. Sampling: **Stratified Split**으로 학습/검증셋 균형 확보

2. Augmentation: 데이터 부족 클래스 대상
Copy-Paste 및 **Oversampling** 적용

탐색적 데이터 분석 (EDA Insight) – 무결성

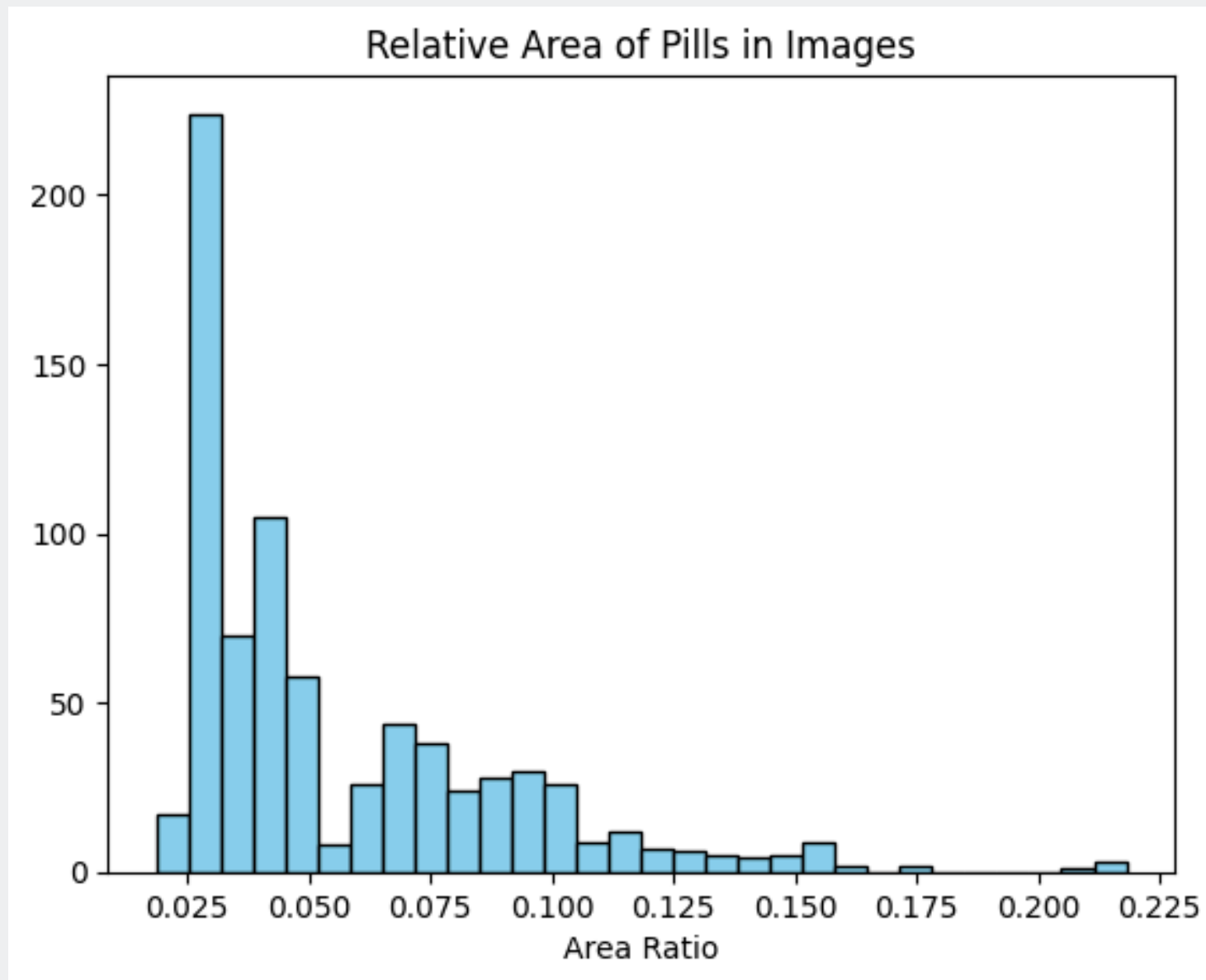


- Invalid Box 1건 및 Missing Box 8건
- 전략

데이터 가치 보존을 위해 삭제 대신 **직접 라벨링** 수행

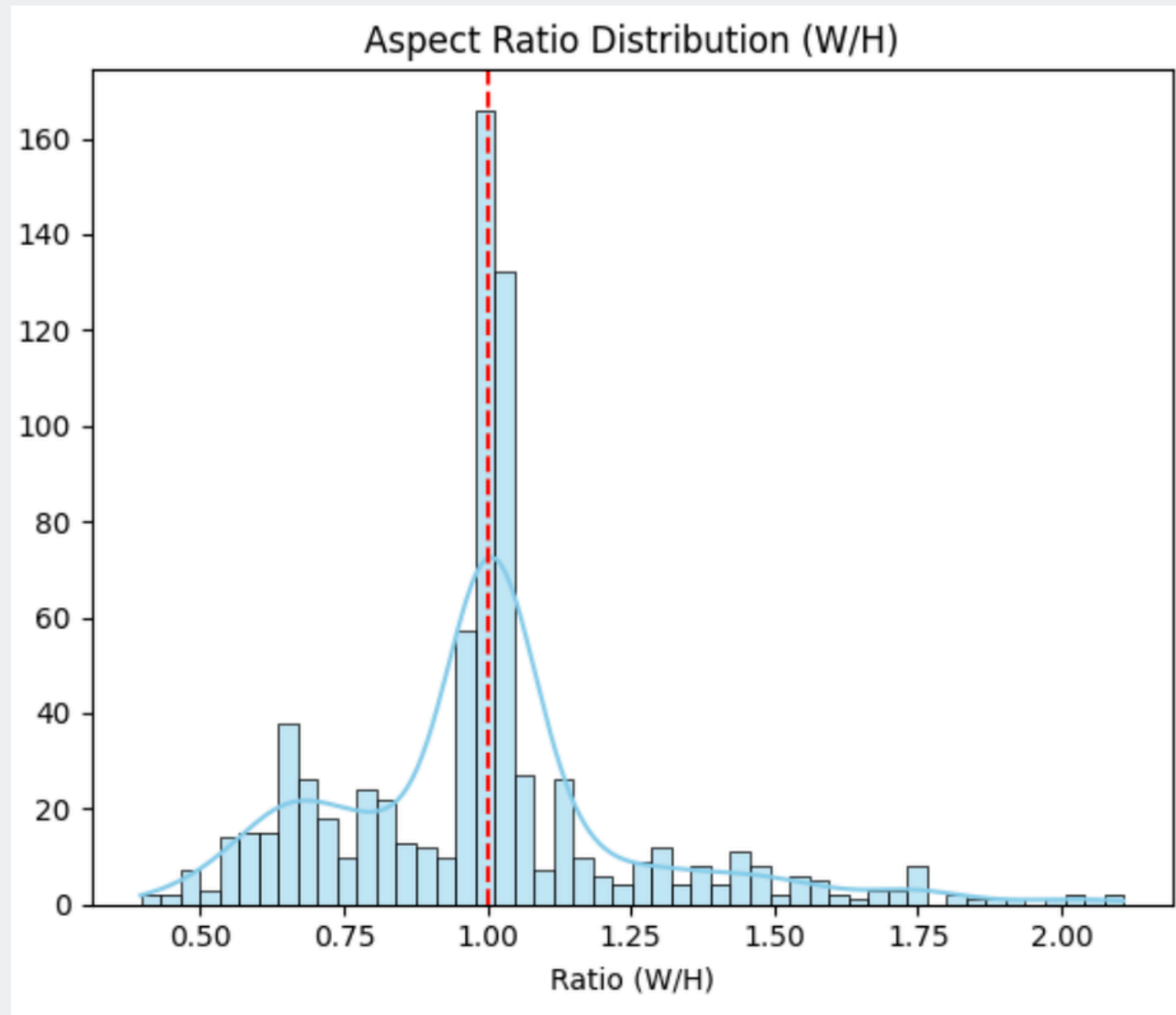
- **좌표값 산출**: 이미지 내 객체 경계면을 기준으로 좌표값 정밀 추출
- **JSON 구조화**: 해당 이미지의 어노테이션 JSON 파일을 직접 수정하여 정규화된 좌표값 업데이트

탐색적 데이터 분석 (EDA Insight) – 객체 크기



- 이미지 대비 객체 면적 비율이 평균 약 5.6% 수준으로 낮음
- 전략
 1. 정보 손실 방지를 위한 **고해상도(960-1080px)** 정하기
 2. **Mosaic** Augmentation 적용

탐색적 데이터 분석 (EDA Insight) – 객체 모양



- 알약 이미지 특성상 형태가 매우 유사하여 변별력 확보 필요
- 장방형 비율이 존재
- 전략

객체 식별력 강화(또렷한 이미지)를 위한 **CLAHE** 기법 도입
다양한 배치 각도에 대한 검출력을 위해 **Rotation/Flip**

베이스라인 모델 선정

아키텍처 탐색 및 모델 선정

1. 후보 모델 비교

Faster R-CNN

2-Stage Detector
높은 검출 정밀도
추론 속도가 느림

YOLO

1-Stage Detector
속도와 정확도의 최적 밸런스
모바일 환경 최적화 용이

RT-DETR

Transformer
뛰어난 성능
높은 학습 리소스 소모

2. YOLO 선정 근거

실시간성

헬스케어 앱의 특성상 즉각적인 알약 식별을 위한 빠른 추론 속도 확보 가능

효율성

적은 파라미터로도 높은 성능을 내며, 빠른 실험 반복에 최적화

확장성

앞선 EDA 전략(Mosaic, Augmentation)을 기본 지원하며 온디바이스 배포에 유리

베이스라인 1.0

1. Baseline 1.0 개요

- 모델 구조: **YOLOv8n**
- 주요 설정:
 - 입력 해상도: 640
 - 전처리: 기본 이미지 (Raw Data)
 - 데이터 분할: Stratified Split (계층적 분할)
 - 증강(Augmentation): 기본 설정
- 핵심 지표: mAP50 기준 약 0.90224, 캐글 점수 약 0.70166

베이스라인 1.0

2. Baseline 선정 이유

(1) 데이터 본연의 특징 파악

원본 데이터셋이 가진 객체(약제)의 특징과 검출 난이도를 객관적으로 파악하기 위한 기준으로 설정함.

(2) 모델 성능의 하한선 확보

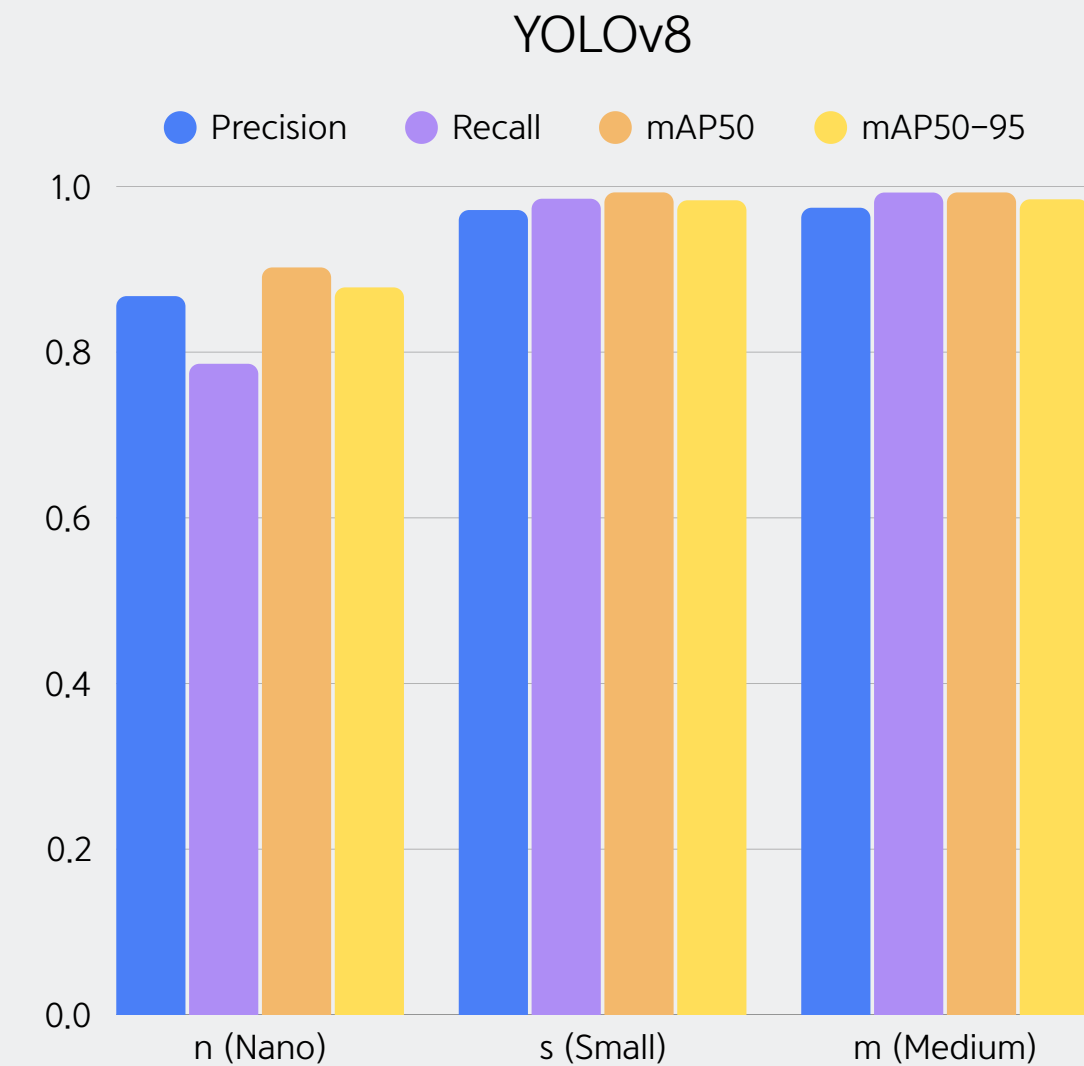
별도의 튜닝 없이도 mAP50-95에서 0.98 이상의 높은 성능을 기록함으로써, 데이터셋의 라벨링 상태가 양호하며 모델이 객체의 위치를 안정적으로 학습하고 있음을 증명함.

YOLOv8 체급 확장

모델 용량 증대에 따른 성능 변화 관찰을 위해 v8 체급별 비교 실험 진행함

Nano 대비 Small 체급에서 급격한 상승이 확인됨

이후 체급을 s로 확장하여 실험을 진행함

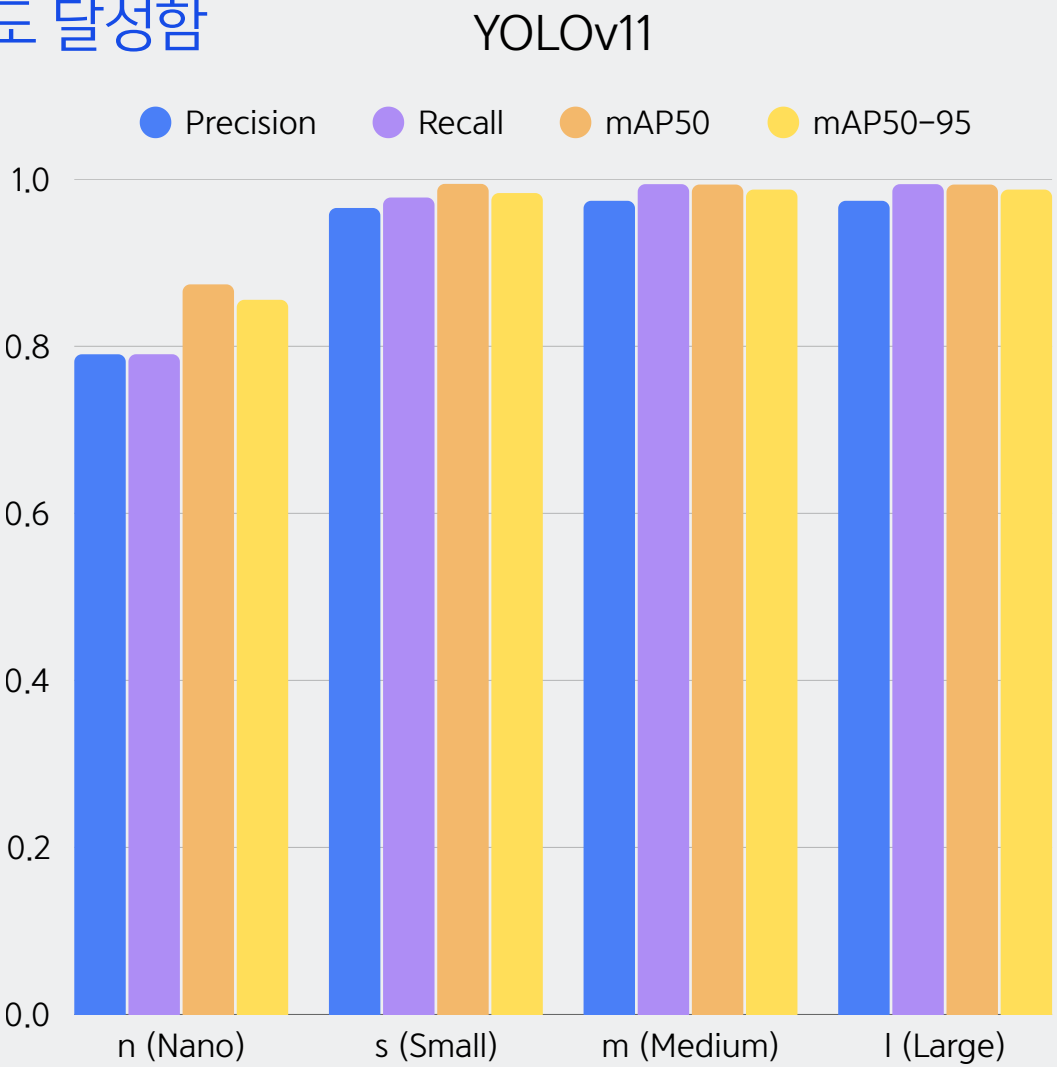


최적 아키텍처 전환 및 최적 체급 도출 (v8s → v11s)

YOLOv8 vs YOLOv11 차이

- YOLOv8: 가장 대중적이고, 다양한 환경에서 안정적인 성능 보장
- YOLOv11: PSA(어텐션) 탑재로 소형/복잡 객체 탐지력 극대화, v8 대비 적은 연산으로 고정밀도 달성함

비교 모델	선정 여부	주요 선정 사유
YOLOv8s	X	안정적이나 최신 아키텍처 대비 특징 추출력 열위
YOLOv11n	X	경량화에 치중되어 복잡한 클래스 식별력 부족
YOLOv11s	O	정확도(Accuracy)와 속도(Latency)의 최적 균형점 달성
YOLOv11m	X	성능 향상 폭 대비 연산 비용 과다 발생



베이스라인 2.0

1. Baseline 2.0 개요

- 모델 구조: **YOLO11s**
- 주요 설정:
 - 입력 해상도: 960
 - 전처리: 기본 이미지 (Raw Data)
 - 데이터 분할: Stratified Split (계층적 분할)
 - 증강(Augmentation): 희귀 클래스 증강 (Copy Paste)
- 핵심 지표: mAP50 기준 약 0.9704, 캐글 점수 약 0.96455

베이스라인 2.0

2. Baseline 선정 이유

(1) 모델 아키텍처의 성능 검증

약제의 미세한 각인이나 형태 차이를 구분해내는 Precision을 높이기 위해 더 깊은 레이어를 가진 모델 스케일을 선택함.

(2) 소수 클래스 탐지 성능 강화

특정 약제의 빈도가 낮은 '클래스 불균형' 문제를 해결하고자 Copy-Paste 증강을 적용함.

Fail Fast

Fail Fast

23	Basemodel 1.0	basemodel_results_PillaTech_team04_02.csv	YOLO v8n (Baseline)	Data cleaned	50	Stratified Split (계층적 분할)	Baseline: YOLO v8n	640	Yolo 기본증강	-	0.70166	수진_Exp 1-2: Stratified (Full)와 똑같은 실험인데 캐글 점수와 차이가 나는게 이상함	49	0.90224	0.87815	측정 안함		
27	hanbyeol_epx1_8s		한별_exp1_YOLOv8s	적용	50	계층적 분할	YOLO v8s	640	Yolo 기본증강	-		베이스모델과 비교 (yolov8n vs yolov8s)		0.99284	0.98331			
27	hanbyeol_exp1_8m		한별_exp1_YOLOv8m	적용	50	계층적 분할	YOLO v8m	640	Yolo 기본증강	-		베이스모델과 비교 (yolov8n vs yolov8m)		0.99279	0.9845			
28	Sooq_yolov8l_v5_local.csv		숙현_v5_YOLOv8l	적용	50	계층적 분할	YOLO v8l	640	Yolo 기본증강	-	0.92471	베이스모델과 비교 (yolov8n vs yolov8l): imgsz = 640, Epoch = 50, 계층적 분할, 배치 = 4이고 TTA 적용 안한 baseline 1.0 용 test.py(이미지 역매핑기법) 사용						
29			한별_exp1_YOLOv11n	적용	50	계층적 분할	YOLO v11n	640	Yolo 기본증강(?)	-								
29			한별_exp1_YOLOv11s	적용	50	계층적 분할	YOLO v11s	640	Yolo 기본증강(?)	-								
30	Hanbyeol_base_yolov11m.csv		한별_exp1_YOLOv11m	적용	50	계층적 분할	YOLO v11m	640	Yolo 기본증강(?)	-	0.88479	3/30 한별: yolov11 medium 결과입니다						
30	Hanbyeol_base_yolov11l.csv		한별_exp1_YOLOv11L	적용	50	계층적 분할	YOLO v11L	640	Yolo 기본증강(?)	-	0.89829	3/30 한별: yolov11 Large 모델 학습 결과입니다. yolov11 Medium 모델보다 높은 성능이 나와서 정확히 캐글 점수로 비교해보고 싶습니다						
24	hanbyeol_v2_copy_paste	v3_augmented_results_PillaTech_team04_03.csv	한별_exp2_copy_paste	적용	50	계층적 분할	Yolo v8n	640	회귀 클래스 증강 (Copy Paste)	-	0.89682	50에폭도 학습, 회귀 클래스 Copy-Paste 증강 후 모델 학습 완료		0.97942	0.95175			
24	hanbyeol_v4_yolov8s	v4_yolov8s_results_PillaTech_team04_04.csv	한별_exp2_YOLOv8s	적용	50	계층적 분할	Yolo v8s	640	회귀 클래스 증강 (Copy Paste)	-	0.83256 (50에폭 전 약 10에폭 결과)	회귀 증강 + v8n vs 회귀 증강 + v8s (50에폭을 다 돌리지 못해서 성능이 떨어진 것 같다)		0.98	0.97	-		
24	hanbyeol_v4_tta	v4_yolov8s_results_tta_team04_06.csv	한별_v4_2	적용	50	계층적 분할	Yolo v8s	640	회귀 클래스 증강 (Copy Paste) + TTA	TTA	0.90361 (50에폭 전 약 10에폭 결과)	회귀 증강 + v8s + TTA (50에폭을 다 돌리고 했더라면 더 높은 점수가 나왔으리라 예상)						
26	hanbyeol_v5_with_tta	hanbyeol_v5_with_tta.csv	한별_v5	적용	50	계층적 분할	yolov8s	1024	회귀 클래스 증강(Copy Paste) + 실시간 온라인 증강 (mosaic=1.0, mixup=0.1, scale=0.5, label_smoothing=0.1)	TTA	0.94569	회귀 클래스 증가 + 고해상도(imgsz:1024) + 실시간 학습 증강 + yolov8s + tta		0.98	0.97			
30	hanbyeol_copy_box_cls_default		한별_box&cls_default	적용	50	계층적 분할	yolo v11s	640	회귀 클래스 증강(Copy Paste v2)		0.87052	box → default: 7.5, cls → default:0.5		0.97716	0.96439	0.95605	0.97826	
30	hanbyeol_box12.0_cls0.5		한별_box&cls (12.0, 0.5)	적용	50	계층적 분할	yolo v11s	640	회귀 클래스 증강(Copy Paste v2)			box → 12.0, cls → 0.5 (bbox를 더 정확하게 그리도록 수정)		0.9711	0.9594	0.9594	0.9142	0.9798

재현성 문제

Train/test 분할방식

복합변수로 비교검증 불가!

TTA

"TTA를 넣으면 반드시 성능이 좋아질 것이다." 라는 가설

해상도, 배치사이즈, 하드웨어 등의
사이즈 축소

학습 단계 최적화

학습 단계 최적화 희귀 클래스 증강

[Problem] 전체 클래스 중 17개 클래스가 단 3개의 샘플만 보유
→ 학습 데이터 부족으로 인한 **희귀 알약의 탐지 성능 저하** 우려

[Strategy] 희귀 클래스 증강 기법 적용

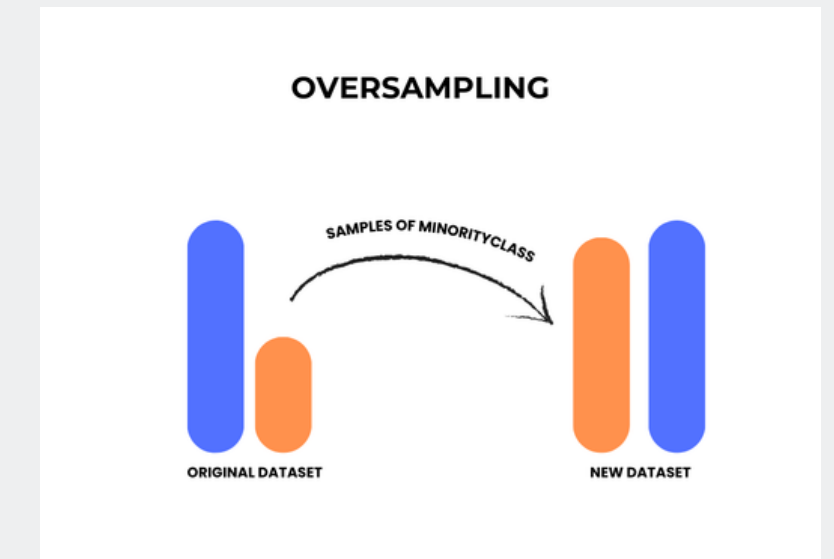
→ **OverSampling**: 단순 중복 활용

→ **Copy-Paste**: 알약 객체를 무작위 배경에 합성

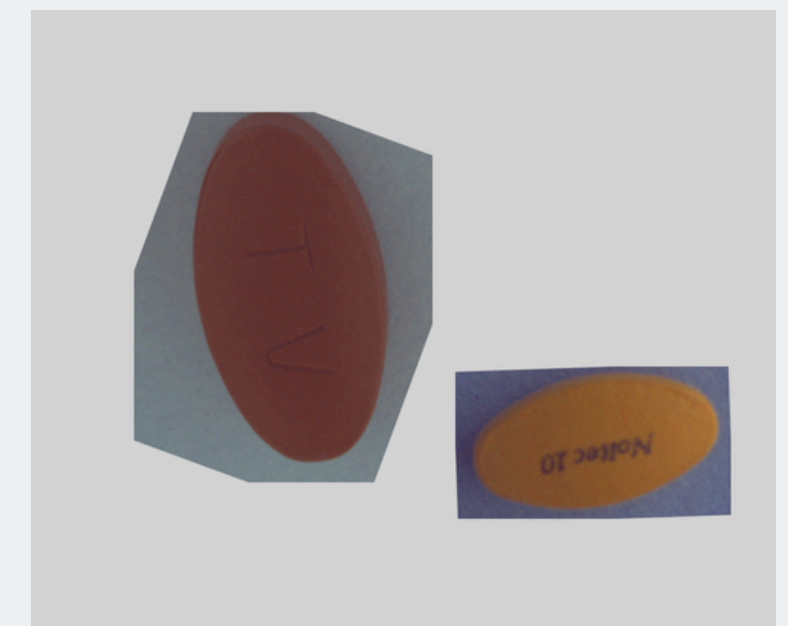
	Precision	Recall	mAP50	mAP50-95
미적용	0.9503	0.9805	0.979	0.9633
OverSampling	0.95203	0.97976	0.97373	0.96305
CopyPaste	0.9721	0.993	0.9931	0.9867

(미적용 캐글: 0.92235 → Copy-Paste 적용 캐글: **0.95967**)

실험 결과 모든 증강 기법 중 가장 압도적인 성능 향상 폭을 기록한 'Copy-Paste' 전략을 프로젝트 최우선 순위로 채택함



<OverSampling>



<Copy-Paste>

학습 단계 최적화 hsv_v 명도/밝기 튜닝

촬영 환경(조명, 그림자 등) 변화에 대한 모델의 강건성 확보를 위해 명도 변화 범위 추적화 진행

hsv_v	mAP50-95	Kaggle Score	비고
0.4	0.9707	0.93634	리더보드 점수 우세
0.6	0.9741	0.92808	내부 지표 최고점 달성
0.8	0.9672		과도한 증강으로 성능 저하

YOLOv11s + Copy-Paste 기반 기초 실험

과도한 명도 증강(0.6 이상)은 조명 노이즈를 유발하여 오히려 실전 탐지력을 저하시키는 것으로 판단됨.
적절한 수준의 증강(0.4)이 테스트 데이터셋의 조명 환경과 가장 높은 유사성을 보이며 안정적인 성능을 유지함 확인됨.

학습 단계 최적화 Optimizer

[목표] Yolo 11s 자동설정된 **AdamW**의 최적성 여부를 Adam계열(Adam, Adam W, RAdam 등)이 아닌 외 다른 계열의 옵티마이저 (SGD)로 우선 비교 검증

[검증] 모든 파라미터 통제, 옵티마이저만 SGD로 변경 vs 옵티마이저별 추천 세팅값과 함께 변경
→ **Learning rate (lr0)**: ADamW의 추천 세팅값으로 SGD로는 학습이 충분히 이루어지지 않음

Optimizer	lr0	lrf	weight_decay	momenntum	캐글점수
ADamW	0.000167	0.01	0.0005	0.9	0.96455
RAdam	0.000167	0.01	0.0005	0.9	-
SGD	0.000167	0.01	0.0005	0.9	0.6091
SGD + 추천값	0.01	-	-	0.937	0.94536

학습 단계 최적화 CLAHE 적용

CLAHE(Contrast Limited Adaptive Histogram Equalization)란?

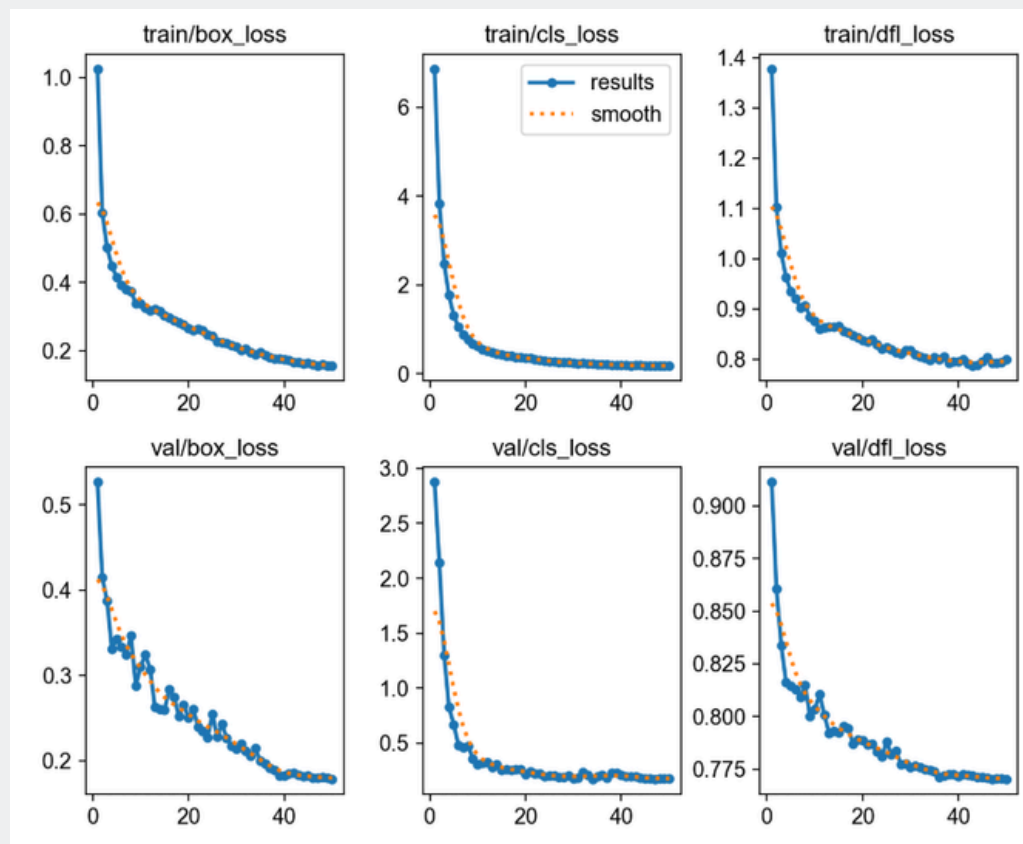
이미지의 대비를 향상시키는 기법 중 하나로, 일반적인 히스토그램 평탄화(HE)의 단점을 보완한 알고리즘



	Precision	Recall	mAP50	mAP50-95	캐글점수
clahe 적용	0.9768	0.9911	0.9948	0.9907	0.95902
clahe 미적용	0.9735	0.9914	0.9941	0.9872	0.95898

- 본 모델에서는 CLAHE를 통해 약제 각인 및 형태의 대비를 높여 성능 향상을 확인
- 단, Mosaic 및 HSV 변화가 적용된 환경에서는 CLAHE의 고정적인 대비 보정이 증강 효과를 상쇄하거나 노이즈를 유발할 수 있음을 확인하여, 실험을 통해 우리 모델에 가장 적합한 조합을 선택함

학습 단계 최적화 Loss Weight 튜닝



- cls_loss: 학습 초기 급격한 하락 후 조기 안정화 양상을 보임
- box_loss & dfl_loss: 50 에폭까지 완만하지만 지속적인 하향 곡선을 그리며 수렴함
- 분석 결과: **위치 추적 정확도**의 추가 개선 여지가 존재함

(box, cls, dfl)	Precision	Recall	mAP50	mAP50-95
(7.5, 0.5, 1.5)	0.95605	0.97826	0.97716	0.96439
(8.5, 0.5, 1.5)	0.9444	0.9798	0.976	0.9664
(7.5, 0.5, 2.0)	0.9505	0.9807	0.9776	0.9581

- mAP50-95에서 유일하게 베이스라인에 비해 상승한 (8.5, 0.5, 1.5) 값이 가장 유의미함
- 다만, precision, recall, mAP50에서는 낮은 성능을 보이므로 추가 실험이 필요함

베이스라인 2.0 기반으로 재실험

단순 반복 실험이 아닌, 자원의 효율적 배분과 결과의 정밀도를 모두 잡기 위한 단계별 최적화 전략을 수립했음.

구분	Stage 1: 저해상도 탐색 (640)	Stage 2: 고해상도 검증 (960)
역할	효율적 가설 검증 및 후보군 압축	최종 성능 판단 및 실전 배포 기준
목적	빠른 학습 속도를 활용한 대규모 파라미터 탐색	실제 제출 환경에 최적화된 성능 도출
비고	유망 조합(Loss, Augmentation 등) 선별	선정된 후보군의 해상도 적응성 및 최종 점수 확정

학습 단계 최적화 hsv_h 색상 변형

중간 강도 hue 증강의 효과

hsv_h=0.020은 색 변화에 대한 강건성을 확보하면서, 과도한 색 왜곡은 피한 값으로 해석됨

국소 최적점 존재

0.020 부근이 상위권이며, 0.030 구간은 하락 신호를 보여 과증강 리스크가 있음을 보여줌.

순위 (Kaggle)	실험명	hsv_h	Precision	Recall	mAP50	mAP50-95	Kaggle
1	Exp20-H3	0.02	0.9709	0.9667	0.9935	0.9919	0.97154
2	Exp19-A (No-HSVH)	0	0.9631	0.9642	0.9929	0.992	0.96678
3	Exp15-Baseline2.0	0.015	0.9704	0.9761	0.9918	0.9897	0.96455
-	Exp20-H2	0.01	0.9655	0.9649	0.9934	0.9915	미제출
-	Exp20-H7	0.025	0.9726	0.9749	0.9934	0.9911	미제출

학습 단계 최적화 Mosaic 변형

가설: box/cls/df1 loss gain 조정으로 Exp22-H3M0 대비 일반화 성능이 개선될 수 있다.

설정: Exp22-H3M0 조건 고정(960, AdamW, seed=42) + loss gain만 변경

결과: 로컬 mAP50-95는 소폭 상승(LBD1 +0.0008)했지만, Kaggle은 기준(0.97199)보다 하락(0.96930)

해석: 640에서 유망했던 gain 조합이 960 환경으로 그대로 전이되지 않았다.

결론: Loss gain 튜닝은 보류하고, 제출 기준 모델은 Exp22-H3M0 유지

향후: 학습 하이퍼튜닝보다 추론 최적화(WBF/TTA) 실험에 집중

순위 (Kaggle)	실험명	mosaic	Precision	Recall	mAP50	mAP50-95	Kaggle
1	Exp21-M0	0	0.9659	0.9707	0.9945	0.9916	0.96894
2	Exp15-Baseline2.0	1	0.9704	0.9761	0.9918	0.9897	0.96455
3	Exp21-M2	0.6	0.9747	0.9648	0.9939	0.992	0.96154
-	Exp21-M1	0.4	0.9727	0.9622	0.9918	0.9897	미제출

학습 단계 최적화 Loss weight 변형

가설: box/cls/df1 gain 조정으로 Exp22-H3M0 대비 성능 향상을 기대
실험 설계: Exp22 조건 고정(960, AdamW, seed42), loss gain만 변경
로컬 결과: LBD10이 mAP50-95 +0.0008로 최고 (0.9915 → 0.9923)
실전 결과: Kaggle은 기준 Exp22(0.97199)보다 하락 (LBD1: 0.96930)
결론: Loss gain 튜닝은 전이 실패, 제출 기준은 Exp22 유지 후 추론 최적화(WBF)로 전환

순위(Kaggle)	실험명	box, cls, dfl	Precision	Recall	mAP50	mAP50-95	Exp22-H3M0와 비교	Kaggle
1	Exp22-H3M0 (기준)	(7.5, 0.5, 1.5)	0.9753	0.963	0.9935	0.9915	0	0.97199
2	Exp25-LBD1	(8.5, 0.5, 2.0)	0.9741	0.9856	0.9941	0.9923	0.0008	0.9693
3	Exp15- Baseline	(7.5, 0.5, 1.5)	0.9704	0.9761	0.9918	0.9897	-0.0018	0.96455
-	Exp25-LD1	(7.5, 0.5, 2.0)	0.9787	0.9599	0.9944	0.992	0.0005	미제출
-	Exp25-LB1	(8.5, 0.5, 1.5)	0.973	0.9632	0.9932	0.9919	0.0004	미제출

추론 성능 극대화

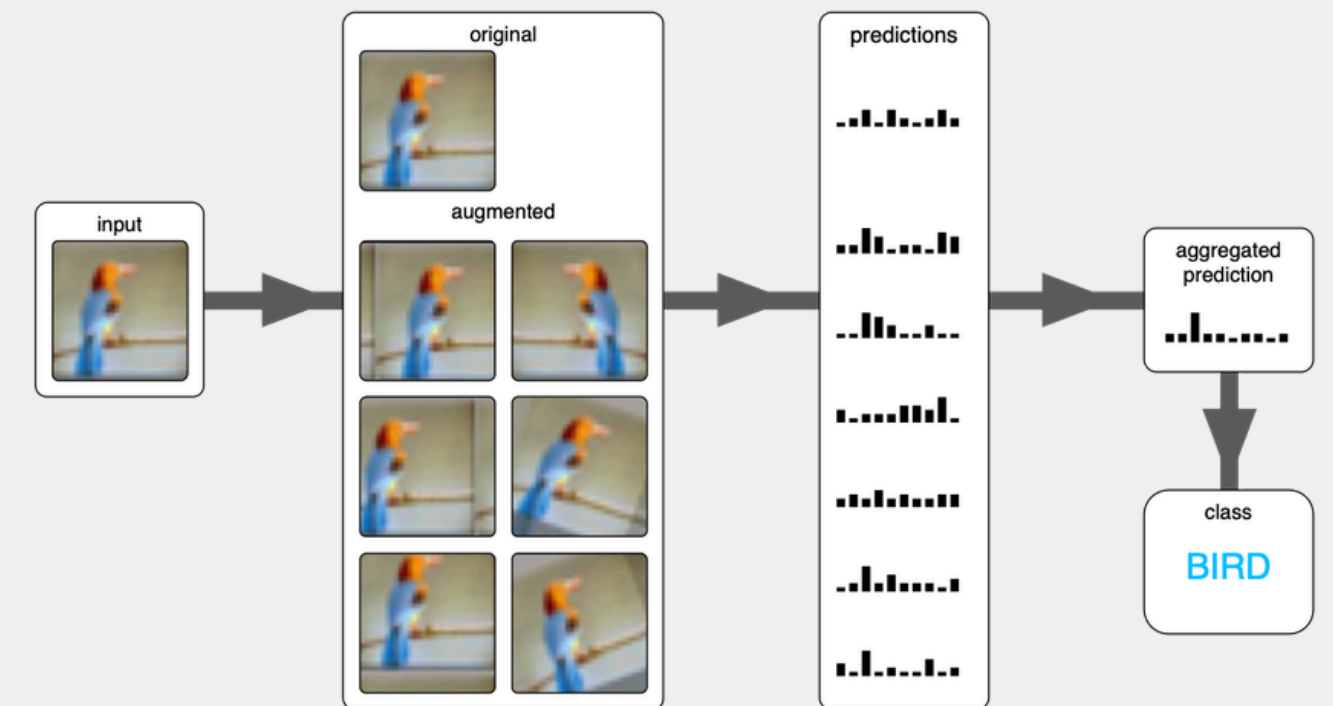
추론 성능 극대화

TTA(Test Time Augmentation)

추론 단계에서 입력 이미지를 여러 방식으로 변형하여 예측 성능을 높이는 기법

- 적용 변형
 - Flip
 - Multi-scale

	캐글점수
TTA 적용	0.94215
TTA 미적용	0.97199



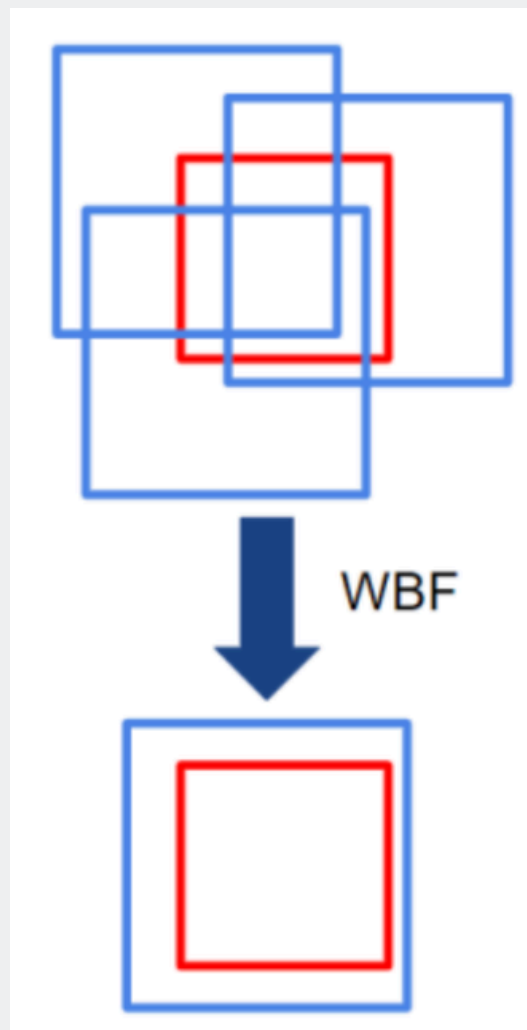
추론 성능 향상을 위해 TTA를 도입하였으나, 미적용 시보다 캐글 점수가 하락하는 결과를 보임

단순히 여러 장을 보는 것이 항상 정확도 향상으로 이어지지 않음을 확인하고, 본 프로젝트 데이터셋과의 부적합성을 분석함
Multi-scale 적용에 따른 미세 특징 정보 왜곡 및 박스 통합 과정의 좌표 오차 발생이 성능 하락의 주요인으로 분석됨

추론 성능 극대화

WBF (Weighted Boxes Fusion)

겹치는 모든 박스의 정보를 가중 평균하여 새로운 박스를 만드는 방식



- 특징: 신뢰도가 높은 박스에는 높은 가중치를, 낮은 박스에는 낮은 가중치를 주어 좌표 계산.
- 장점: 여러 예측값의 "평균"을 내기 때문에, 단일 박스보다 실제 객체의 위치에 훨씬 더 가깝게 박스가 형성됨.

구분	실험명	Precision	Recall	mAP50	mAP50-95	Kaggle
기준	Exp22-H3M0	0.9753	0.963	0.9935	0.9915	0.97199
1차 WBF	Exp26 (Exp22 + 640/960/1024 WBF)	-	-	-	-	0.97345
2차 WBF	Exp27 (42/123/777 WBF)	-	-	-	-	0.98077

결론 및 향후 과제

결론

1. 실험을 통한 최적의 아키텍처 및 설정 도출

- **모델 체급의 황금 밸런스:** YOLO11s 모델로 캐글 0.98077이라는 고득점을 달성하며, 실전 성능과 실험 효율의 최적점을 찾아 최적의 모델 효율을 증명함.
- **해상도와 일반화의 상관관계:** 해상도 증대가 반드시 선형적인 캐글 점수 상승으로 이어지지 않음을 발견했음. 이는 해상도 변화가 증강 효과 및 객체 스케일과 상호작용하기 때문이며, 실전 데이터 분포에 최적화된 960 해상도를 최종 채택했음.

2. 데이터 중심 최적화의 유효성

- **라벨링 정제의 힘:** 9장의 Bounding Box 오류를 수동으로 JSON 좌표 보정한 결과, 데이터 손실 없이 학습 데이터셋의 완전성을 확보하여 모델의 신뢰도를 높였음.
- **증강 기법의 명암:** Copy-Paste와 hsv_h 조정은 희귀 클래스 학습과 색상 변화 대응에 유효했으나, Mosaic과 CLAHE 실험을 통해 과도한 전처리가 오히려 실전 환경에서의 일반화 성능을 저해할 수 있다는 '과증강 리스크'를 실증했음.

향후 과제

1. MLOps 환경 구축 및 실험 관리 자동화

- **실험 트래킹 도입:** 이번 프로젝트에서 수많은 실험을 관리하며 겪은 어려움을 해결하기 위해, WandB나 MLflow를 도입하여 하이퍼파라미터와 메트릭을 실시간으로 시각화하고 비교하는 환경을 세팅하고자 함.
- **데이터 및 모델 버전 관리(DVC):** 데이터 정제 과정과 모델 가중치 변화를 버전별로 관리하여, 언제든지 특정 시점의 실험을 복구하고 협업할 수 있는 엔지니어링 파이프라인을 고도화할 예정

2. 엄격한 재현성 검증 체계 구축

- **Seed-Level 변동성 제어:** 실험 과정에서 발생했던 '동일 설정, 타 성능' 문제를 해결하기 위해, CUBLAS_WORKSPACE_CONFIG 설정 및 고정 시드 기반의 Deterministic Training 파이프라인을 완벽히 구축할 계획임.
- **통계적 유의성 확보:** 단일 시드 결과에 의존하지 않고, Multi-seed(3~5회) 반복 실험을 통한 평균 성능 및 표준편차를 기록하여 모델의 신뢰도를 객관화할 계획임.

Thank you!

PillaTech (4팀)의 발표를 들어주셔서 감사합니다.